Next-Generation SIEM Architecture for Connected and Autonomous Vehicles in Intelligent Transportation Systems

Muhammad Uzair and Abdul Sami
SAMOVAR, Télécom SudParis, Institut Polytechnique de Paris, Palaiseau, France

***Abstract*—Connected and Autonomous Vehicles (CAVs) generate large volumes of Vehicle-to-Everything (V2X) data, yet no validated Security Information and Event Management (SIEM) system exists to monitor this domain. We present a three-layer SIEM architecture for Intelligent Transportation Systems. Layer 1 collects Basic Safety Messages (BSMs) from vehicles via DSRC and C-V2X collectors. Layer 2 collects Signal Phasing and Timing (SPaT) data from Roadside Units via Syslog-NG and Filebeat. Both layers stream data through Apache Kafka to the Layer 3 SIEM backend, which uses the ELK Stack for normalisation, storage, and correlation. In parallel, a Python-based ML scoring script consumes BSM data from the same Kafka topic and runs a two-stage classifier a binary XGBoost model first determines whether an attack is present, and if so, a multi-class LightGBM model identifies the attack group. The SIEM correlation results and the ML predictions are then compared to reduce false positives. The models are trained on the VeReMi Extension dataset and tested against real Tampa CV Pilot BSM and SPaT data. We also introduce seven novel functions covering cross-layer validation, temporal scoring, RSU fingerprinting, spatial clustering, SHAP explainability, SHAP–LIME false positive filtering, and counterfactual explanations.**

I. Introduction

Connected vehicles are becoming a standard part of modern transportation. These vehicles constantly exchange safety messages with each other and with roadside infrastructure. A Connected Autonomous Vehicle broadcasts a Basic Safety Message ten times every second, reporting its position, speed, heading, and acceleration. At the same time, Roadside Units at intersections send out Signal Phasing and Timing messages that describe the current state of traffic signals. In a city-wide deployment, this can easily produce millions of messages per second.

This volume of real-time data creates a serious security monitoring challenge. In traditional enterprise networks, Security Information and Event Management systems handle exactly this kind of problem by collecting logs from many sources, correlating them, and raising alerts when something looks wrong. However, a recent systematic review of 111 published papers on CAV cybersecurity [1] found that no one has actually built and validated a working SIEM for the ITS domain. The closest work is V2X-SIEM by Abreu et al. [2], which proposes an ELK-based architecture but relies entirely on hand-written rules and has only been tested on simulated data.

The threat landscape facing connected vehicles is broad. We identified five categories of attacks from the literature: data poisoning and adversarial evasion targeting ML models, GPS spoofing and sensor manipulation targeting localisation, network-level attacks like Sybil and denial-of-service targeting V2X communications, CAN bus injection targeting in-vehicle systems, and large-scale infrastructure attacks like ghost vehicles targeting the transport network as a whole. Despite these threats, no end-to-end SIEM has been validated for ITS. The research community agrees that the SIEM belongs at the backend, not inside the vehicle [2]–[4]. The idea is that lightweight collectors on the vehicle side forward data to a centralised backend where the heavy analysis happens. But no published system has actually implemented this model with real V2X data and machine learning detection together.

Our work addresses this gap. We designed and built a complete three-layer SIEM that collects real V2X data, streams it through Kafka, stores and correlates it in the ELK Stack, and simultaneously runs a machine learning detection pipeline in a separate Python script. The ML pipeline uses a two-stage approach first a binary XGBoost classifier decides if a message is an attack, then a multi-class LightGBM classifier identifies what kind of attack it is. The results from both the SIEM correlation engine and the ML pipeline are compared, which helps reduce false positives. We also propose seven novel functions that add capabilities no existing ITS security system provides. In this work, we set out to answer three questions:

1) Where should the SIEM sit in the ITS ecosystem?
2) How should vehicle and RSU data reach the SIEM?
3) What ML approach can detect attacks while keeping false positives low?

The remaining sections of this paper are structured as follows. Section II surveys related work and identifies the research gap our system addresses. Section III presents the proposed three-layer architecture and the parallel ML detection pipeline. Section IV describes the dataset selection and feature engineering. Section V details the two-stage classification pipeline. Section VI introduces the seven novel detection and explainability functions. Section VII describes the end-to-end SIEM workflow. Section VIII discusses the filtering strategy. Section IX presents the demonstration and experimental results. Section X maps the cyber attack landscape to our architecture coverage. Section XI compares our system

with existing solutions. Section XII concludes the paper with perspectives for future work.

II. Related Work

A. SIEM Systems for Connected Vehicles

We reviewed 14 papers across IEEE, Springer, ACM, Elsevier, and arXiv. Table I lists the most relevant ones.

V2X-SIEM [2] is the only published SIEM built specifically for connected vehicles. It uses Elasticsearch and Kibana, achieves 2–26 ms alert latency, and can handle over 100 events per second. But it has no machine learning at all, it was only tested on simulated traffic, and it runs as a single node with no scalability story. Grimm et al. [3] were the first to formally argue that SIEM belongs at the backend for automotive security, but they stopped at the concept and never built anything. EVSOAR [5] combines a SIEM with automated response using XGBoost, but it targets EV charging stations, not V2X. SeMaCoCa [6] applies self-learning anomaly detection on connected car backend data but does not do any cross-layer correlation between vehicle and infrastructure sources. Table II compares these systems.

TABLE I
ITS-related SIEM research papers (authentic and ranked).

#	Paper Title	Pub.	Year	Quartile	SIEM?
1	V2X-SIEM [2]	IEEE	2025	Scopus	Backend
2	Network Sec. Mon. [3]	Springer	2020	Scopus	Backend
3	Vehicular Comms [7]	Elsevier	2020	Q1 SJR 1.56	N/A
4	V2X Survey [8]	ACM	2023	Q1 SJR 5.80	N/A
5	Elastic Stack [9]	IEEE	2020	Scopus	Backend
6	CAN Anomaly [10]	ACM	2019	Scopus	N/A
7	CAV Review [1]	IEEE	2025	Q1 SJR 0.9	Gap found

TABLE II
SIEM and AI integration comparison across reviewed systems.

System	SIEM Tool	AI Model	Real	XAI	FP
V2X-SIEM	ELK	None	×	×	×
VSOC [11]	ELK	Future	×	×	×
SeMaCoCa [6]	Custom	Self-learn	×	×	×
Conn. Cars [4]	Backend	HMM+Regr.	×	×	×
EVSOAR [5]	ELK+SOAR	XGB+FL	×	×	×
Ours	**ELK+Kafka**	**XGB+LGBM**	✓	✓	✓

Real=real-world data, XAI=explainability, FP=FP filtering.

B. AI Models for Vehicular Intrusion Detection

We looked at 14 ML models used for vehicle security. Table III summarises them. The most relevant is MTH-IDS [12], which uses a stacking ensemble of Decision Tree, Random Forest, Extra Trees, and XGBoost, reaching 99.99% accuracy on the CAN-intrusion dataset. This confirmed that XGBoost is a strong choice for vehicular anomaly detection, which is why we chose it for our binary stage. The same group extended this with LCCDE [13], achieving 99.9997% F1-score. For the multi-class stage, we compared XGBoost and LightGBM head-to-head and found that LightGBM trained roughly twice as fast while matching accuracy within 0.2%, so we adopted LightGBM for production.

TABLE III
AI models reviewed for SIEM integration.

Model	AI Type	Key Feature	Result	Tier
MTH-IDS [12]	Hybrid ens.	DT+RF+ET+XGB	99.99%	1
LCCDE [13]	Supervised	XGB+LGBM+Cat	99.9997%	1
XAI-FL [14]	FL + XAI	DNN+SHAP+Krum	V2X first	1
CANintelli [15]	Deep Learn.	CNN+Attn-GRU	93.79%	2
TCAN-IDS [16]	Deep Learn.	Temporal Conv.	3.4ms GPU	2
GCN-2-F [17]	Supervised	GCN+Transf.	Spat-temp	2
VAE-CNN [18]	Unsuperv.	VAE+CNN slide	Real-time	2
Aloqaily [19]	Supervised	RF,XGB,LGBM	99.9%	3
MaREA [20]	Supervised	Multi-class RF	0.9997	3

C. Explainable AI for IDS

Gaspar et al. [21] showed that SHAP and LIME can be applied to intrusion detection systems. The XAI-FL-IDS framework [14] was the first to use these techniques specifically for connected vehicles. Pinto et al. [22] proposed checking whether SHAP and LIME agree on the top features as a way to filter out false positives, reporting a median Cohen's kappa of 0.94. Table IV lists the key papers.

TABLE IV
Papers supporting explainability functions.

XAI Feature	Paper	Result
SHAP+LIME	Gaspar et al. [21]	Proved on MLP-based IDS
SHAP+LIME V2X	Taheri [14]	First for connected vehicles
SHAP VeReMi	Bangui et al. [23]	Position most influential
SHAP-LIME FP	Pinto et al. [22]	Median κ=0.94
Counterfactual	Jiang et al. [24]	First for network IDS
XAI Survey	Charmet et al. [25]	Counterfactuals emerging

D. Gap Analysis

TABLE V
Gap analysis comparing existing systems.

System	SIEM	ML	Real Data	X-Layer	XAI	FP Filt.	C.Fact.	Kafka
V2X-SIEM [2]	✓	×	×	×	×	×	×	×
Grimm [3]	C	×	×	×	×	×	×	×
EVSOAR [5]	✓	✓	×	×	×	×	×	×
SeMaCoCa [6]	✓	✓	×	×	×	×	×	×
MTH-IDS [12]	×	✓	×	×	×	×	×	×
Papadakis [26]	✓	R	P	×	×	×	×	×
Ours	✓	✓	✓	✓	✓	✓	✓	✓

✓=yes, ×=no, C=concept only, R=rules only, P=partial.

As Table V shows, none of the existing systems satisfy all eight requirements. Ours is the first to combine a full SIEM with ML detection on real V2X data, cross-layer correlation,

explainability, false positive filtering, and Kafka streaming in one system.

III. Proposed Architecture

The system has three layers. Fig. 1 shows how they connect.

A. Layer 1: V2V + Internal Vehicle

Each vehicle has an On-Board Unit that reads sensor data from the CAN bus and builds a BSM packet following the SAE J2735 standard. Two collectors handle this:

- **DSRC Collector:** receives BSM packets via UDP, parses the binary structure, applies basic filtering (schema validation, duplicate removal, range checks), and publishes to the Kafka topic `v2x-bsm`.
- **C-V2X Collector:** does the same thing for messages arriving over the cellular V2X radio, publishing to the same topic.

The key fields from the Tampa CV Pilot BSM dataset are: `coreData_id`, `coreData_lat`, `coreData_long`, `coreData_speed`, `coreData_heading`, and `coreData_accelset_long`.

B. Layer 2: V2I (RSU Side)

Each RSU broadcasts SPaT messages at 10 Hz describing traffic signal states. Syslog-NG collects these logs and Filebeat forwards them to the Kafka topic `v2x-spat`. The key fields from the Tampa SPaT dataset are: `metadata_RSUID`, `metadata_generatedAt`, `IntersectionState_id_id`, `IntersectionState_revision`, and `MovementState`.

C. Communication Layer: Apache Kafka

Kafka sits between the collectors and the backend. We use two topics: `v2x-bsm` for vehicle messages and `v2x-spat` for RSU messages. Kafka was the obvious choice because V2X data rates are far beyond what HTTP or TCP can handle reliably. Kafka can sustain millions of messages per second with sub-10 ms latency, and if the SIEM goes down briefly, Kafka retains the messages so nothing is lost.

The important design decision here is that Kafka supports multiple consumers on the same topic. Both the SIEM pipeline (through Logstash) and our Python ML scoring script subscribe to `v2x-bsm` at the same time. This means the ML inference runs completely independently from the SIEM ingestion. Neither blocks the other.

TABLE VI
End-to-end communication flow.

Source	Collector	Kafka Topic	Consumer	Dest.
OBU (DSRC)	DSRC Coll.	v2x-bsm	Logstash	ES
OBU (C-V2X)	C-V2X Coll.	v2x-bsm	Logstash	ES
OBU (both)	Same as above	v2x-bsm	Python ML	ML Out
RSU	Syslog-NG + FB	v2x-spat	Logstash	ES

ES=Elasticsearch, FB=Filebeat.

D. Layer 3: SIEM Backend

The SIEM backend runs on the ELK Stack and processes data in six steps:

1) **Normalisation + Store:** Logstash receives data from Kafka, converts it into JSON, and stores it in Elasticsearch.
2) **Aggregation:** three novel functions compute additional features (trust scores, RSU deviation scores, cluster anomaly scores) and append them to the stored data.
3) **Correlation:** the SIEM correlation engine matches BSM events with SPaT events using the shared RSU ID and timestamp, checking for inconsistencies.
4) **Comparison with ML Results:** the correlation output is compared against the classification from the Python ML script. When both agree, the alert is confirmed. When they disagree, the event is flagged for review.
5) **Monitoring:** confirmed alerts appear on the Kibana dashboard.
6) **Report Generation:** each alert report includes a SHAP explanation of why the event was flagged, a counterfactual showing which field was attacked, and the ML confidence score.

E. Parallel ML Detection (Python Script)

The ML scoring script (`ml_scorer.py`) runs as a separate long-lived process. It subscribes to the `v2x-bsm` Kafka topic with its own consumer group (`ml-scorer-v3`) and processes every incoming BSM through a two-stage cascade:

Stage 1 — Binary XGBoost: the 17 raw BSM features are normalised using a pre-fitted StandardScaler and passed to an XGBoost binary classifier. This model achieves 100% accuracy on VeReMi because the noise fields (`pos_noise`, `hed_noise`) have completely non-overlapping distributions between benign and attack records. If the prediction is benign, the message is labelled as such and sent to Elasticsearch. No further processing is needed.

Stage 2 — Multi-Class LightGBM: if Stage 1 flags an attack, the scorer computes 10 delta features (differences between consecutive messages from the same vehicle) and 4 sliding-window features (standard deviations and message rate over the last 10 messages). The full 31-feature vector is normalised and passed to a LightGBM classifier that identifies the attack group. The scorer maintains a per-vehicle deque of size 10 in memory to compute these temporal features.

The output (attack label, attack group, severity, and confidence score) is written to a daily Elasticsearch index (`v2x-scored-YYYY.MM.DD`) where Kibana dashboards can display it alongside the SIEM correlation results.

IV. Dataset Selection and Feature Engineering

A. Tampa CV Pilot BSM Dataset (Vehicle Side)

The Tampa CV Pilot BSM dataset [27], published by the US Department of Transportation, contains 34,464 real BSM records across 49 columns following the SAE J2735 standard.

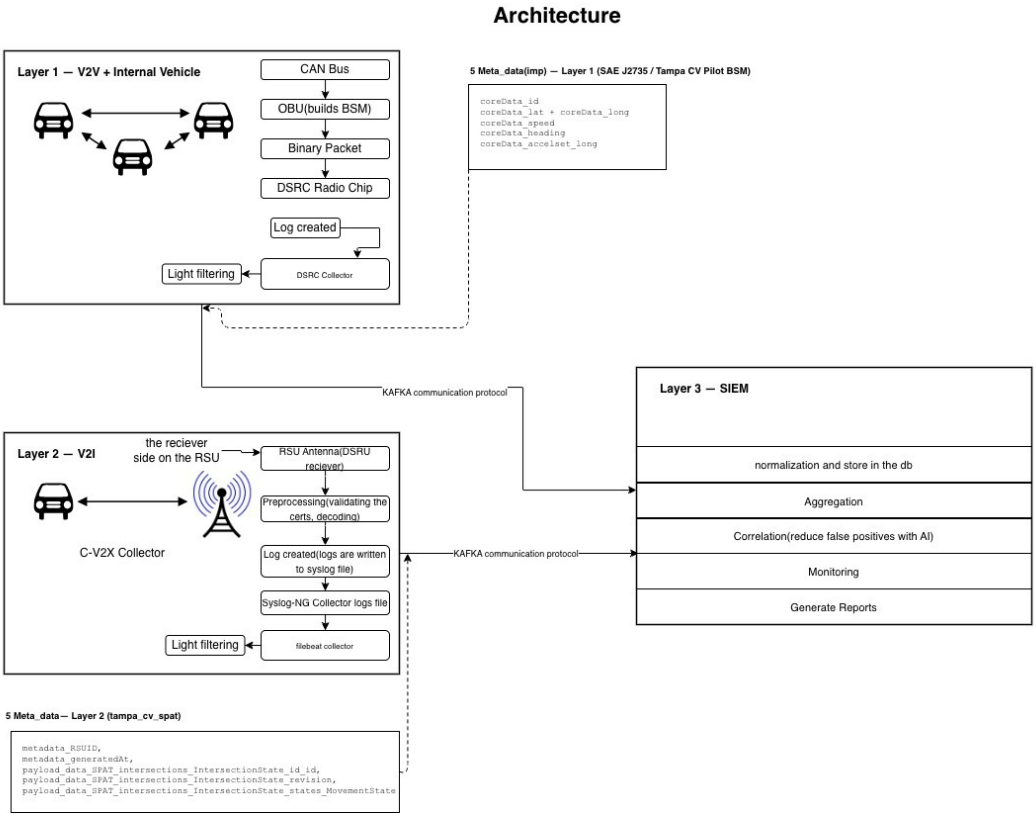

Fig. 1. Three-layer SIEM architecture for ITS/CAV. Layer 1 collects BSMs from vehicles, Layer 2 collects SPaT from RSUs, both stream through Kafka to the Layer 3 SIEM backend and the parallel ML scoring script.

B. Tampa CV Pilot SPaT Dataset (RSU Side)

The Tampa CV Pilot SPaT dataset [28] consists of SPaT messages broadcast by RSUs at 10 Hz. This provides real-world infrastructure data for cross-validation with vehicle claims.

C. VeReMi Extension Dataset (Attack Training)

We train our models on the VeReMi Extension dataset [29], which contains 2,269,540 labelled BSM records across 20 classes (1 benign + 19 attack types), perfectly balanced at 113,477 records per class. The dataset was generated through 225 VEINS simulations covering position falsification, speed manipulation, denial of service, replay, and other attack types. Its BSM fields follow the same SAE J2735 structure as the Tampa dataset, so a model trained on VeReMi can be deployed on Tampa data.

D. Feature Engineering

We use three layers of features. The 17 raw features come directly from each BSM record: timestamp, position (X/Y), position noise, speed (X/Y), speed noise, acceleration (X/Y), acceleration noise, heading (X/Y), and heading noise. A key insight is that the noise fields are near zero for benign vehicles but large and structured for attackers, which is why the binary classifier achieves perfect accuracy.

On top of the raw features, we compute 10 delta features by taking the difference between consecutive messages from the same vehicle: changes in time, position, speed, heading, distance moved, expected distance from reported speed, and the error between actual and expected distance. Finally, we compute 4 sliding-window features over the last 10 messages per vehicle: standard deviations of position, speed, and heading, plus the message rate. These temporal features are essential for distinguishing attack types that only become visible over a sequence of messages.

E. Feature Mapping: VeReMi to Tampa BSM

After removing the simulation artefact columns (noise injection records that do not exist in real data) and the label columns, 8 input feature columns remain that map directly between VeReMi and Tampa. Table VII shows this mapping. Training on the noise columns would give artificially high accuracy on VeReMi but would fail completely on real Tampa data.

TABLE VII
FEATURE MAPPING FROM VEREMI TO TAMPA BSM.

VeReMi	Tampa BSM	Description
rcvTime	metadata_recordGeneratedAt	Timestamp
pos_0	coreData_long	Longitude
pos_1	coreData_lat	Latitude
spd_0	coreData_speed	Speed
spd_1	coreData_accelset_lat	Lateral speed
acl_0	coreData_accelset_long	Long. accel.
acl_1	coreData_accelset_lat	Lateral accel.
hed_0	coreData_heading	Heading
hed_1	coreData_accelset_yaw	Yaw rate

TABLE VIII
DROPPED VEREMI COLUMNS (SIMULATION ARTEFACTS).

Dropped Column	Reason
pos_noise_0/1	Attacker-injected position noise absent in real data
spd_noise_0/1	Attacker-injected speed noise absent in real data
acl_noise_0/1	Attacker-injected acceleration noise absent in real data
hed_noise_0/1	Attacker-injected heading noise absent in real data
type	Filtered to BSM only, then dropped
attack	Binary label kept separately, not an input feature
attack_type	Multiclass label kept separately, not an input feature

F. Attack Class Grouping

The original VeReMi dataset has 19 attack types. Our first attempt at multi-class classification with all 19 classes only achieved 32% accuracy. Analysis of pairwise feature distances showed that many classes are nearly indistinguishable from a single receiver's perspective. We merged them into 6 behavioural groups: DoSFamily (8 types including DoS, Sybil, and Disruptive variants), PositionAttack (4 types), SpeedManip (4 types), ReplayAttack, DelayedMessages, and EventualStop. This grouping improved accuracy from 32% to 69%.

V. AI MODEL: TWO-STAGE CLASSIFICATION PIPELINE

A. Model Architecture

The detection engine is a cascade, not a parallel system. The binary model runs on every message, and the multi-class model runs only on messages that the binary model already flagged as attacks. This saves computation and prevents the multi-class model from ever labelling benign traffic with an attack type.

Stage 1 — Binary XGBoost: trained on a balanced subset of VeReMi (113,477 benign + 113,477 attack records) using the 17 raw features. Hyperparameters: 5000 estimators with early stopping at 100 rounds, max depth 6, learning rate 0.1. Achieves 100% test accuracy with zero false positives and zero false negatives.

Stage 2 — Multi-Class LightGBM: trained on attack-only data balanced across the 6 groups, using all 31 features (17 raw + 10 delta + 4 window). Hyperparameters: 5000 estimators with early stopping, 127 leaves, max depth 8, learning rate 0.05, class weight balanced. Achieves 68.85% weighted F1-score. LightGBM was chosen over XGBoost after a head-to-head comparison showed it trained approximately twice as fast while matching accuracy.

B. Justification

We chose this two-stage XGBoost + LightGBM design for six reasons:

1) **Speed:** both models run in sub-millisecond time per event, fast enough for real-time Kafka stream processing.
2) **False positive reduction:** the binary gate ensures only confirmed attacks reach the multi-class stage. Comparison with SIEM correlation adds a second reduction layer.
3) **SHAP compatibility:** both XGBoost and LightGBM support SHAP TreeExplainer natively.
4) **Class imbalance:** XGBoost's `scale_pos_weight` and LightGBM's `class_weight` handle imbalanced data.
5) **Literature validation:** XGBoost is the most widely used model in vehicular IDS research [12].
6) **Deployment simplicity:** a single Python script with no GPU requirement, running on commodity hardware, consuming directly from Kafka.

TABLE IX
AI MODEL COMPARISON — WHY TWO-STAGE XGBOOST + LIGHTGBM.

Model	Accuracy	Speed	SHAP?	Fit?
XGB+LGBM 2-Stage	**100/69%**	**<0.6ms**	**Yes**	**Best**
Stacking Ensemble [12]	99.99%	0.574ms	Yes	Good
CNN+GRU [15]	93.79%	GPU req.	Approx.	No
TCAN-IDS [16]	High	3.4ms GPU	No	No
MLP/Neural	99.9%	38s test	Approx.	No
RF alone	99.95%	Fast	Yes	Partial

TABLE X
PAPERS SUPPORTING TREE-BASED APPROACHES FOR VEHICULAR IDS.

Paper	Models Used	Result
MTH-IDS [12]	DT+RF+ET+XGBoost Stack	99.99%
LCCDE [13]	XGB+LGBM+CatBoost	99.9997%
EVSOAR [5]	XGBoost + Fed. DNN	EV station IDS
Multi-Class IoV [30]	RF+ET+XGBoost	Ensemble best
DRX Ensemble [31]	DT+RF+XGB voting	Voting effective
FFS-IDS [32]	DT base + RF meta	Detection up

C. Accuracy Improvement Journey

Reaching the current accuracy was an iterative process. We started with a 19-class XGBoost model using only raw and delta features, which gave just 32%. Adding the 4 sliding-window features brought it to around 40%. Merging the 19 classes into 8 groups pushed accuracy to 57%. Further merging into 6 groups reached approximately 63%. Finally, increasing the number of estimators from 200 to 5000 with early stopping brought us to the current 69%. The remaining 31% error is largely irreducible with single-receiver features, because some attack distinctions (e.g., DoS vs Sybil) require cross-vehicle correlation that is not available in per-message feature vectors.

VI. NOVEL DETECTION AND EXPLAINABILITY FUNCTIONS

We introduce seven functions that add capabilities no existing ITS SIEM provides. Functions 2–4 run in the Aggregation block, Functions 1, 5–7 run in the Correlation block. Table XI gives the full specification.

TABLE XI
SEVEN NOVEL FUNCTIONS INTRODUCED IN THE SIEM ARCHITECTURE.

#	Function	Block	Description & Novelty
1	BSM–SPaT Cross-Val.	Correl.	Checks BSM speed/position vs SPaT signal phase. *Novel:* no paper cross-validates BSM against SPaT.
2	Temporal Scoring	Aggreg.	Tracks `coreData_id` across messages; builds per-vehicle trust score. *Novel:* no temporal trust in any SIEM.
3	RSU Fingerprinting	Aggreg.	Learns each RSU's normal broadcast pattern; flags deviations. *Novel:* no RSU profiling from SPaT.
4	Spatial Clustering	Aggreg.	Clusters BSMs near intersections; detects individual and group anomalies. *Novel:* no clusters + SPaT used.
5	SHAP Explain.	Corr.+R.	Calculates each feature's contribution %. *Novel:* never cross-layer BSM+SPaT in SIEM.
6	SHAP–LIME FP	Correl.	Top-5 features from both; agree ($\kappa \geq 0.50$) = real, else suppress. *Novel:* no SHAP–LIME FP filter in V2X.
7	Counterfactual	Corr.+R.	Min change to flip prediction identifies attacked field. *Novel:* no counterfactuals in V2X SIEM.

VII. SIEM WORKFLOW

The complete workflow has seven steps (Table XII).

TABLE XII
SIEM WORKFLOW SUMMARY.

Step	Component	Functions	Output
1	Kafka Arrival	None	Raw msgs to SIEM + Python
2	SIEM: Normalise	None	JSON data in ES
3	SIEM: Aggregation	F2, F3, F4	3 derived features
4	SIEM: Correlation	F1	Correlation results
5	Python: ML	XGB + LGBM	Attack label + type
6	Comparison	SIEM vs ML	FP reduction
7	Reports	F5, F6, F7	Alert reports

Step 1: BSM and SPaT data arrive on Kafka. Logstash and the Python ML script both consume `v2x-bsm` in parallel. Only Logstash consumes `v2x-spat`. **Step 2:** Logstash normalises the data into JSON and stores it in Elasticsearch. **Step 3:** The Aggregation block computes derived features per vehicle and per RSU. **Step 4:** The Correlation engine cross-checks BSM events against SPaT events using shared RSU ID and timestamp. **Step 5:** In parallel, the Python scorer runs BSM data through the binary XGBoost gate. If an attack is detected, the LightGBM multi-class model identifies the attack group and assigns a severity level (CRITICAL for DoSFamily, HIGH for PositionAttack and ReplayAttack, MEDIUM for SpeedManip and DelayedMessages, LOW for EventualStop). **Step 6:** The SIEM correlation results and the ML classification are compared. Agreement confirms the alert disagreement flags the event for manual review. **Step 7:** Each confirmed alert report includes SHAP explanation, counterfactual analysis, ML confidence, and cross-layer evidence.

VIII. FILTERING STRATEGY

Filtering happens at two levels. At the vehicle and RSU level, the DSRC/C-V2X collectors and the Filebeat collector filter data before forwarding it to Kafka. Rather than sending everything, they select only the relevant message types (BSM from vehicles, SPaT from RSUs) and apply basic schema validation. At the SIEM level, there is no additional filtering. All events received from both layers are normalised by Logstash and stored in Elasticsearch. The detection and analysis is handled by the correlation engine and the parallel ML pipeline, not by dropping events.

IX. DEMONSTRATION AND RESULTS

To validate the architecture end-to-end, we built a working demonstration using the VeReMi dataset, Apache Kafka, Elasticsearch, Kibana, and the Python ML scoring pipeline.

A. Test Environment

The demonstration runs on a single machine with Kafka, Elasticsearch, and Kibana deployed as Docker containers. The ML scorer (`ml_scorer.py`) runs as a standalone Python process consuming from Kafka. Three data producers were developed for testing: `sumo_kafka_producer.py` generates realistic benign BSMs from a SUMO traffic simulation, `vehicle_simulation.py` replays a balanced mix of 1,000 benign and 1,000 attack records from VeReMi, and `attack_injector.py` injects specific attack types on demand for targeted testing. Fig. 2 in Appendix A shows the ML scoring server at startup.

B. Binary Classification Results

The binary XGBoost classifier achieves 100% accuracy, 100% precision, 100% recall, and an F1-score of 1.00 on the VeReMi test set. The false positive rate and false negative rate are both exactly zero. This perfect performance is a property of the VeReMi dataset: the noise fields have completely non-overlapping distributions between benign and attack records. Fig. 3 in Appendix A shows 361 scored events indexed in Kibana.

C. Multi-Class Classification Results

The LightGBM multi-class classifier achieves a weighted F1-score of 68.85% across the 6 attack groups. Performance varies by group: DoSFamily and EventualStop are well-separated, while SpeedManip and PositionAttack show more overlap. The 31% residual error is attributable to attack-type distinctions that require cross-vehicle correlation not available in per-message features. Fig. 4 in Appendix A shows the Elasticsearch field statistics confirming the ML output fields.

D. End-to-End Pipeline Validation

The scored events are indexed in Elasticsearch under `v2x-scored-YYYY.MM.DD` with fields including `ml_is_attack`, `ml_confidence`, `ml_attack_type`, and `ml_severity`. Kibana dashboards display real-time attack detection, severity distribution, and per-vehicle timelines.

The attack injector tool was used to verify that each of the six attack groups is correctly classified and assigned the appropriate severity level. Figs. 5, 6, and 7 in Appendix A show the V2X simulation dashboard under benign and attack scenarios.

X. Cyber Attack Landscape in ITS

We identified five groups of attacks from the literature. Table XIII shows how our architecture covers each group.

TABLE XIII
CYBER-ATTACK GROUPS AND ARCHITECTURE COVERAGE.

Group	Examples	Coverage
1: AI/Data	Data poisoning, FDI, evasion	XGBoost + SHAP identifies features
2: Localisation	GNSS spoofing, LiDAR replay	Spatial clustering + BSM–SPaT
3: VANET/V2X	Sybil, bogus info, DoS/DDoS	VeReMi model + temporal scoring
4: Intra-Vehicle	CAN injection, ECU bypass	CAN-derived BSM anomalies
5: ITS Infra.	Ghost vehicles, traffic poison	RSU fingerprinting + spatial

XI. Comparison with Existing Solutions

a) V2X-SIEM [2]: is the closest existing work. It uses the same ELK foundation and targets the same V2X domain. But it has no ML, no real data, no cross-layer correlation, and no explainability. We extend it with a working two-stage ML pipeline, real Tampa dataset support, BSM–SPaT cross-validation, and three XAI mechanisms.

b) MTH-IDS [12]: proved that tree-based ensembles work well for vehicular intrusion detection. But it is a standalone IDS with no SIEM integration, no V2X data ingestion, and no cross-layer correlation. We took the XGBoost component from their validated approach and embedded it in a full SIEM pipeline with Kafka streaming and explainability.

c) EVSOAR [5]: uses XGBoost + Federated DNN for EV charging station security. Our system targets a different domain (V2X/ITS), uses a two-stage pipeline instead of a single model, and adds cross-layer BSM–SPaT functions that EVSOAR does not have.

d) Capabilities unique to our proposal.: Three things our system does that no reviewed system does:

1) **BSM–SPaT Cross-Layer Correlation:** we are the first to cross-validate vehicle BSM claims against RSU SPaT data in real time within a SIEM.
2) **Dual False Positive Reduction:** the SIEM correlation and the ML classification run independently and their results are compared. No other system uses this dual-path approach.
3) **Counterfactual Actionable Intelligence:** our system can tell an analyst exactly which field was attacked, not just that an attack occurred.

XII. Conclusion

We built a complete Next-Generation SIEM for connected vehicles that addresses the gap identified by the PRISMA review of 111 CAV cybersecurity papers [1]. The three-layer architecture with Kafka streaming and ELK processing provides a solid, open-source foundation. The two-stage ML pipeline (binary XGBoost at 100% accuracy + multi-class LightGBM at 69% F1) runs in parallel with the SIEM correlation engine, enabling dual-path false positive reduction. The seven novel functions add capabilities that no published ITS security system provides.

For future work, we plan to add cross-vehicle correlation features to improve multi-class accuracy beyond 69%, experiment with larger sliding windows, explore federated learning for privacy-preserving training, and validate the system on a live ITS testbed.

References

[1] H. Fares, N. Faci, S. Bouzefrane, and M. Benaissa, "Cybersecurity in Connected and Autonomous Vehicles: A Systematic Review," *IEEE Access*, pp. 1–38, 2025.

[2] R. Abreu, P. Ferreira, C. Serôdio, F. Branco, A. Valente, and M. J. Reis, "V2X-SIEM: A Lightweight Security Information and Event Management Architecture for Connected and Autonomous Vehicles," in *2025 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA)*, pp. 1–6, IEEE, 2025.

[3] D. Grimm, F. Pistorius, and E. Sax, "Network Security Monitoring in Automotive Domain," in *Future of Information and Communication Conference (FICC)*, vol. 1130, pp. 786–800, Springer, 2020.

[4] E. Gutflash *et al.*, "Advanced Analytics for Connected Cars." arXiv preprint arXiv:1711.01939, 2017.

[5] T. Freitas *et al.*, "EVSOAR: Security Orchestration, Automation and Response via EV Charging Stations." arXiv preprint arXiv:2503.16988, 2025.

[6] B. Glas *et al.*, "POSTER: Anomaly-Based Misbehaviour Detection in Connected Car Backends," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 1–3, ACM, 2017.

[7] M. Bozdal, M. Samie, S. Aslam, and I. Jennions, "Cybersecurity Challenges in Vehicular Communications," *Vehicular Communications*, pp. 1–28, 2020.

[8] R. Sedar, C. Kalalas, F. Vazquez-Gallego, L. Alonso, and J. Alonso-Zarate, "A Survey of Security and Privacy Issues in V2X," *ACM Computing Surveys*, pp. 1–39, 2023.

[9] F. Mulyadi, H. Suhartanto, and W. C. Wibowo, "Implementing Dockerized Elastic Stack for SIEM," in *2020 International Conference on Information and Communication Technology (InCIT)*, pp. 1–6, IEEE, 2020.

[10] G. Madzudzo, K. Harnett, K. Burnham, and J. A. Garcia-Macias, "Context-Aware Anomaly Detector for CAN Bus," in *Proceedings of the 3rd ACM Computer Science in Cars Symposium (CSCS)*, pp. 1–8, ACM, 2019.

[11] V. S. Barletta, D. Caivano, A. Nannavecchia, and M. Scalera, "V-SOC4AS: A Vehicle-SOC for Improving Automotive Security," *Algorithms*, vol. 16, no. 2, pp. 1–19, 2023.

[12] L. Yang, A. Moubayed, and A. Shami, "MTH-IDS: A Multitiered Hybrid Intrusion Detection System for Internet of Vehicles," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 1–17, 2022.

[13] L. Yang, A. Shami, G. Stevens, and S. DeRusett, "LCCDE: A Decision-Based Ensemble Framework for Intrusion Detection in the Internet of Vehicles," in *2022 IEEE Global Communications Conference (GLOBECOM)*, pp. 1–6, IEEE, 2022.

[14] R. Taheri, R. Javidan, and M. Shojafar, "XAI-FL-IDS: Explainable Federated Learning Intrusion Detection System for Connected Vehicles," *Electronics*, vol. 14, no. 3, pp. 1–18, 2025.

[15] A. U. Rehman, S. Ur Rehman, M. Khan, M. Alazab, and T. G. Reddy, "CANintelliIDS: Detecting In-Vehicle Intrusion Attacks on a Controller Area Network Using CNN and Attention-Based GRU," *IEEE Transactions on Network Science and Engineering*, pp. 1–15, 2021.

[16] P. Cheng, K. Xu, S. Li, and M. Han, "TCAN-IDS: Intrusion Detection System for Internet of Vehicle Using Temporal Convolutional Attention Network," *Symmetry*, vol. 14, no. 2, pp. 1–15, 2022.

[17] X. Zhang *et al.*, "GCN-2-Former: A Graph Convolutional and Transformer-Based Model for Spatial-Temporal Intrusion Detection," *Scientific Reports*, vol. 15, pp. 1–18, 2025.
[18] D. Oladimeji, M. Mahmoud, and W. Alasmary, "VAE-CNN for V2X BSM Anomaly Detection," *Applied Sciences*, vol. 15, no. 4, pp. 1–18, 2025.
[19] M. Aloqaily *et al.*, "Machine Learning-Based Misbehavior Detection in V2X Communications," *IEEE Transactions on Intelligent Transportation Systems*, pp. 1–12, 2025.
[20] G. Bovenzi *et al.*, "MaREA: Multi-Class RF Ensemble Approach for Misbehavior Detection in VANETs," in *2024 IEEE International Conference on Communications Workshops (ICC Workshops)*, pp. 1–6, IEEE, 2024.
[21] R. Gaspar, R. Lopes, and J. Magalhães, "Explainable Artificial Intelligence for Intrusion Detection Systems: LIME and SHAP," *IEEE Access*, vol. 12, pp. 1–12, 2024.
[22] A. Pinto *et al.*, "SHAP–LIME Consensus as a False Positive Filter for Network Intrusion Detection Systems," *Annals of Telecommunications*, pp. 1–10, 2025.
[23] H. Bangui and B. Buhnova, "SHAP-Based Feature Analysis for Misbehavior Detection in VANETs Using VeReMi," *PeerJ Computer Science*, vol. 10, p. e2145, 2024.
[24] W. Jiang *et al.*, "Tabular Diffusion-Based Counterfactual Explanations for Network Intrusion Detection," in *Proceedings of the ACM CCS Workshop on Artificial Intelligence and Security*, pp. 1–10, ACM, 2024.
[25] F. Charmet *et al.*, "Explainable Artificial Intelligence for Cybersecurity: A Survey," *Frontiers in Artificial Intelligence*, vol. 8, pp. 1–22, 2025.
[26] N. Papadakis, N. Spanoudakis, K. Giannakis, and C. Karaberi, "SIEM on Automated Minibuses," in *Transport Research Arena (TRA) 2024*, pp. 73–84, Springer, 2024.
[27] U.S. Department of Transportation, "Tampa CV Pilot BSM Dataset." https://data.transportation.gov, 2025.
[28] U.S. Department of Transportation, "Tampa CV Pilot SPaT Dataset." https://data.transportation.gov, 2025.
[29] J. Kamel, M. R. Ansari, J. Petit, A. Kaiser, I. B. Jemaa, and P. Urien, "VeReMi Extension: A Dataset for Misbehavior Detection in VANETs," in *2020 IEEE International Conference on Communications (ICC)*, pp. 1–6, IEEE, 2020.
[30] A. Alsaedi *et al.*, "Multi-Class Intrusion Detection for IoV Using Ensemble of RF, ET, and XGBoost," *IEEE Access*, vol. 11, pp. 1–14, 2023.
[31] T. H. H. Aldhyani and H. Alkahtani, "DRX Ensemble: Decision Tree, Random Forest, and XGBoost Voting for Vehicular Network Intrusion Detection," *Sensors*, vol. 23, no. 8, pp. 1–18, 2023.
[32] M. Zeeshan *et al.*, "FFS-IDS: Feature Fusion Stacking Intrusion Detection System for Internet of Vehicles," *Computer Communications*, vol. 198, pp. 233–247, 2023.

APPENDIX

Fig. 2. ML scoring server startup showing models loaded and Kafka consumer connected.

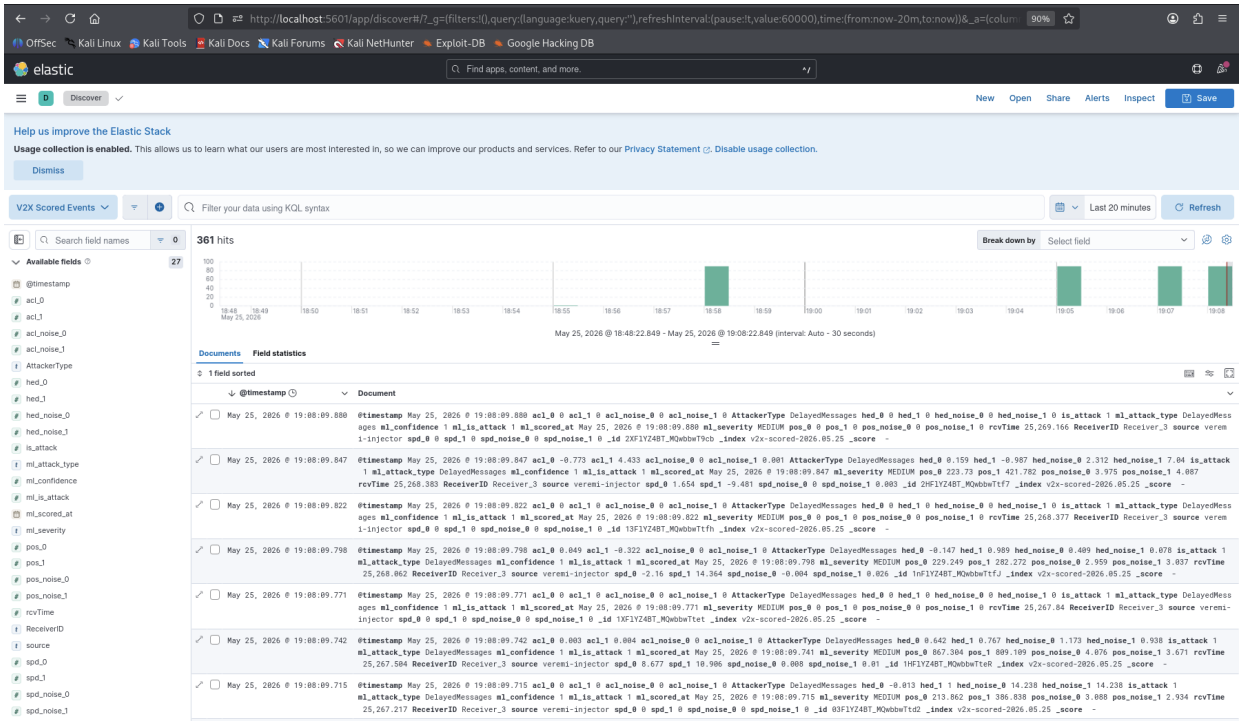

Fig. 3. Kibana Discover view showing 361 scored BSM events with `ml_attack_type`, `ml_severity`, and `ml_confidence` fields indexed in Elasticsearch.

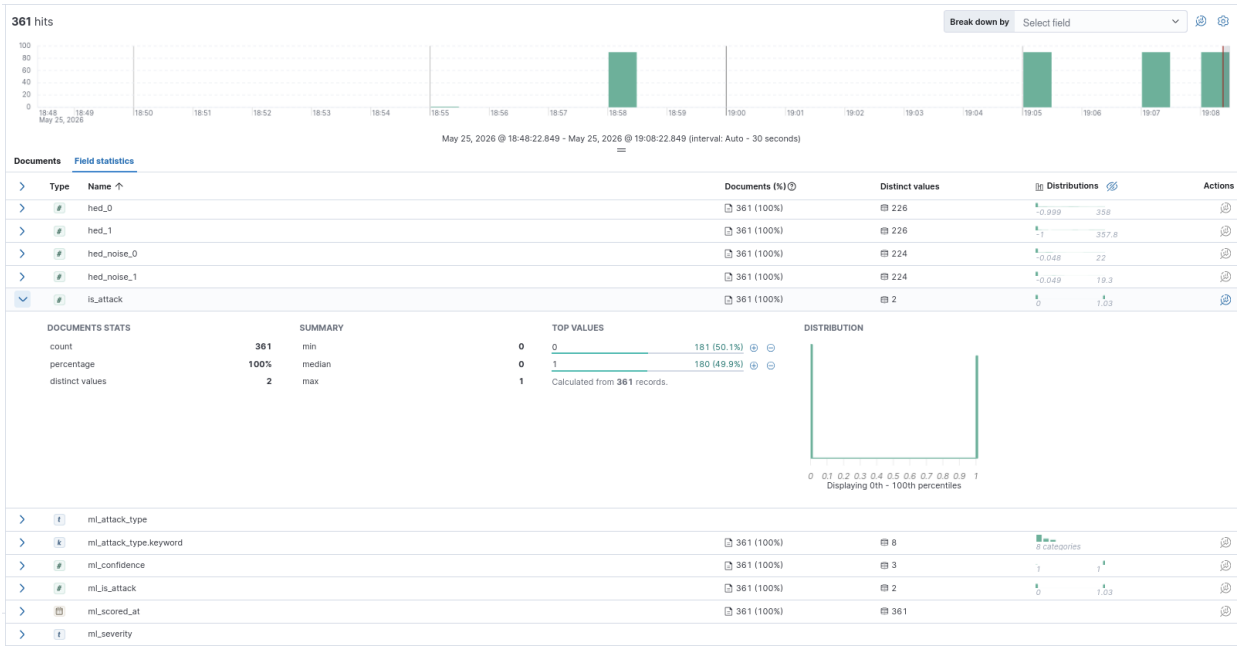

Fig. 4. Elasticsearch field statistics: `is_attack` split 50.1%/49.9%, `ml_attack_type` with 8 distinct values, `ml_severity` with 5 severity categories.

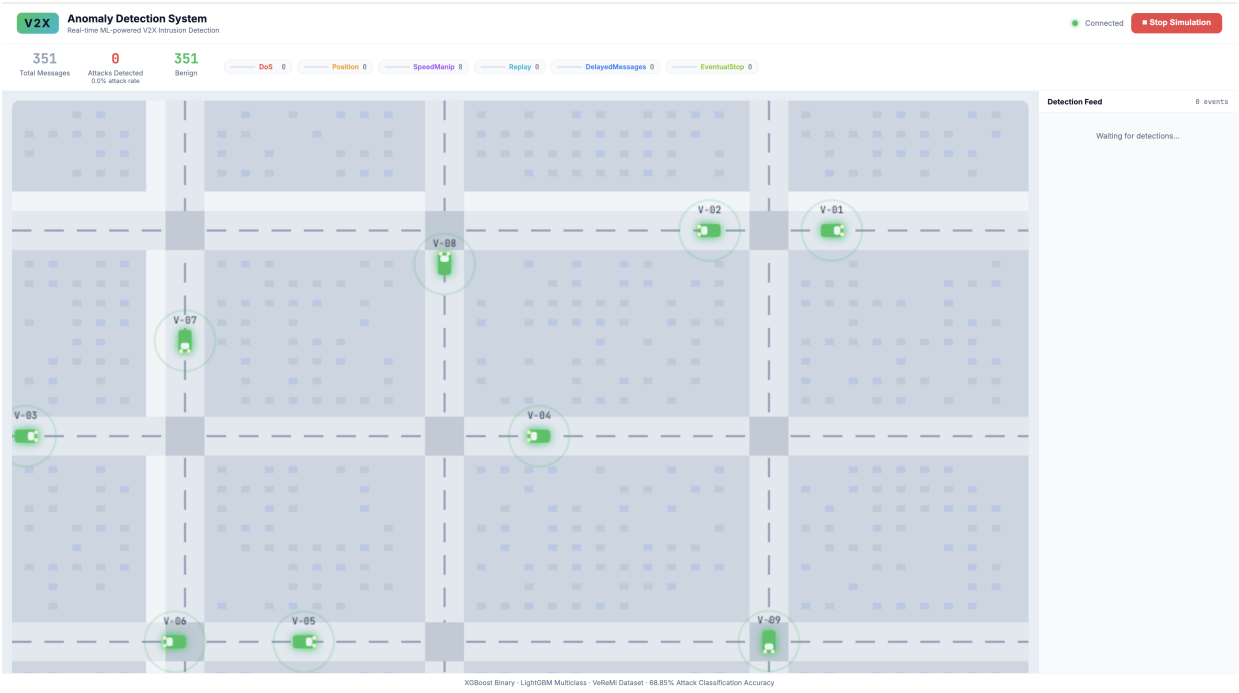

Fig. 5. V2X simulation dashboard under benign scenario: 351 messages processed, 0 attacks detected, all vehicles shown in green.

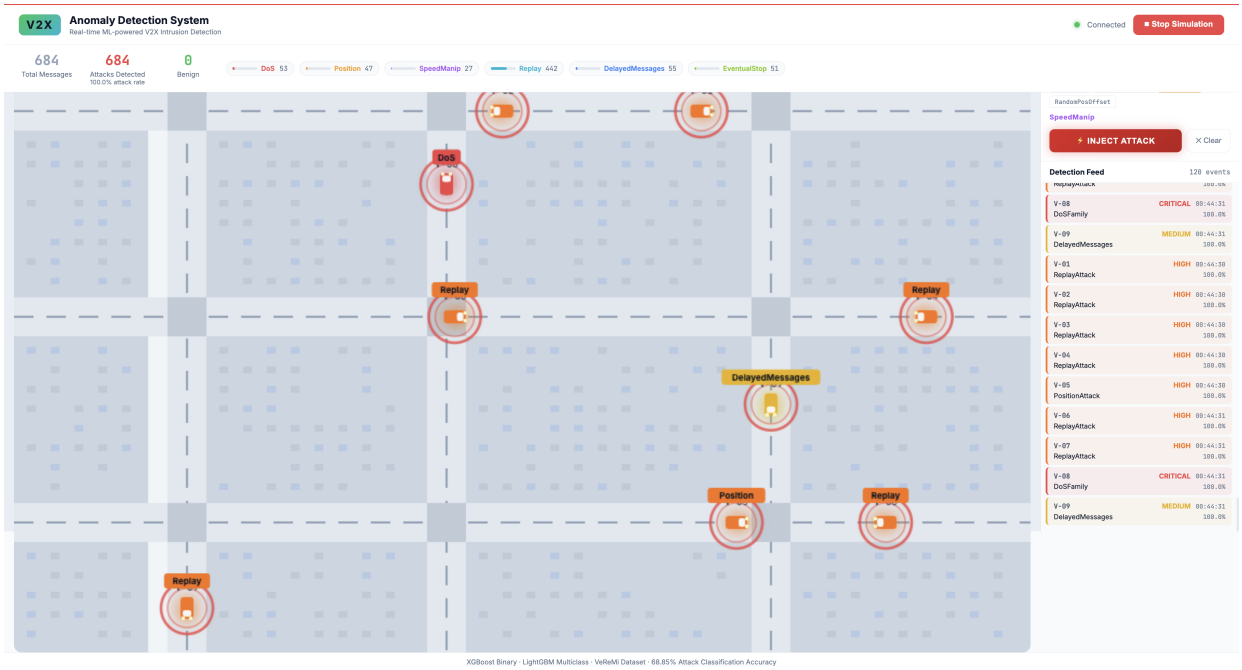

Fig. 6. V2X simulation dashboard under replay attack injection: vehicles flagged red with attack type labels and severity levels displayed in the detection feed.

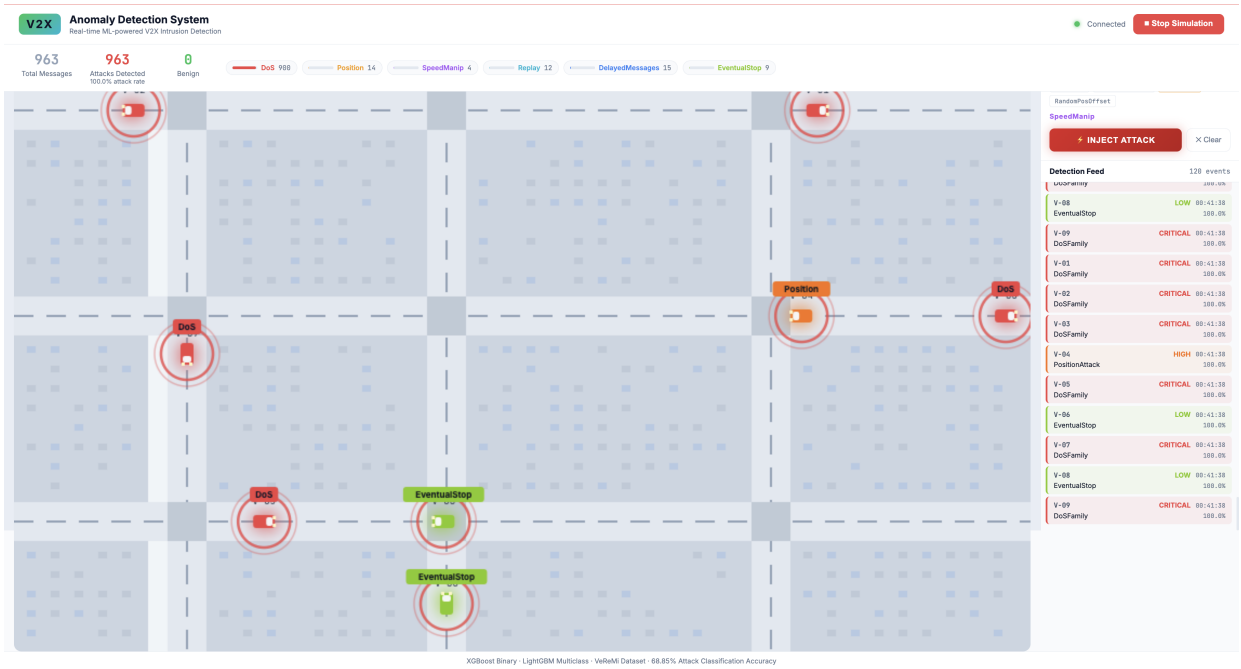

Fig. 7. V2X simulation dashboard under DoS attack injection: 684 messages processed, 684 attacks detected at 100% detection rate, with DoSFamily classified as CRITICAL severity.